AgentLeash

A spending leash for AI agents
on Bitcoin Cash

Solo entry · BCH BLAZE 2026 · Mainnet Track

ttimesai-star.github.io/agentleash-bch
github.com/ttimesai-star/agentleash-bch



AgentLeash

covenant · x402 · WizardConnect

Agent**Leash**!

Give your AI agent a wallet
it **cannot** run away with.

The agent signs its own payments. The Bitcoin Cash network,
not a server and not the agent's code, enforces the budget.

https://ttimesai-star.github.io/agentleash-bch/

The Story of AgentLeash

AI agents now pay for things themselves: data, APIs, compute, per call, over HTTP 402 ("x402").

To pay, an agent needs a key. A hot key is **unlimited**: one prompt injection, one bug, one leaked log, and the wallet is gone.

The usual fix is a custodial API with spending rules. Then you trust a server instead of the agent.

So we asked: can the network itself hold the agent to a budget, while the owner can still pull the plug at any time?

What a compromised agent tries, and who stops it

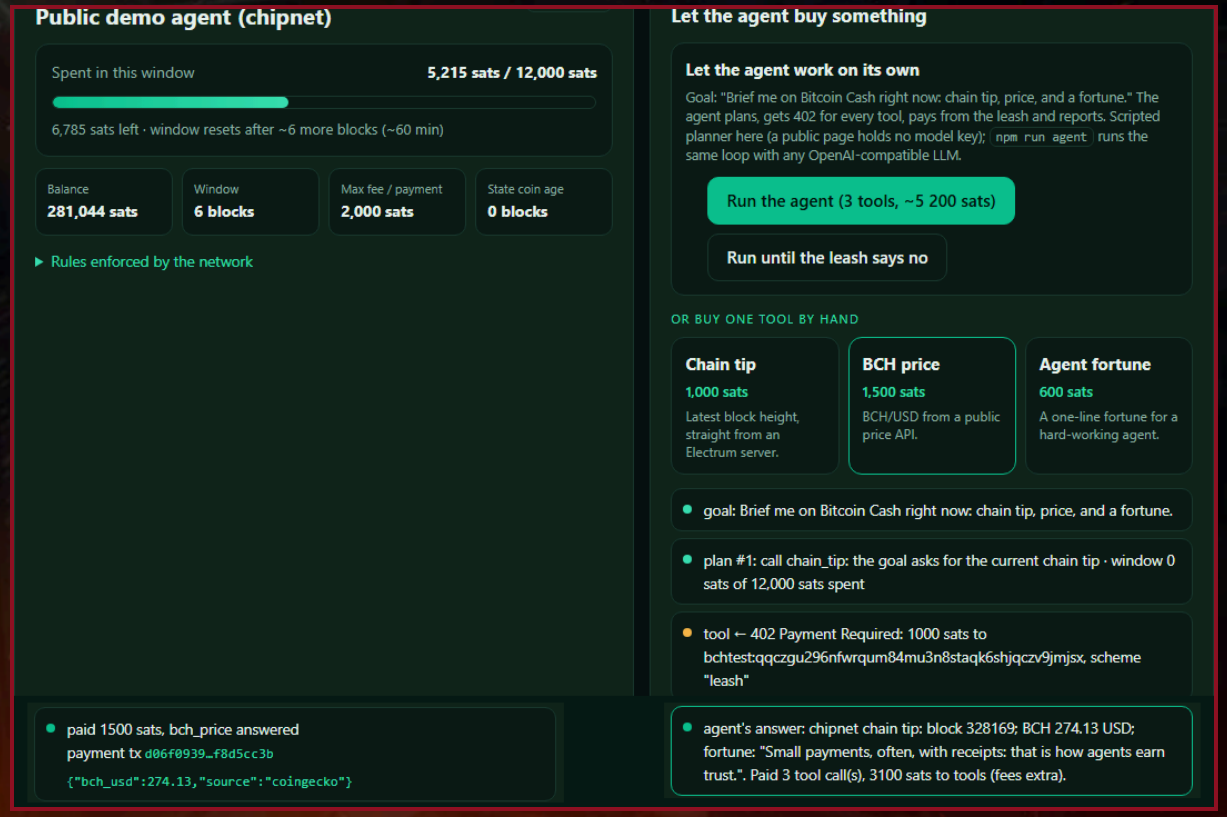
ATTACK	STOPPED BY
pay the attacker	covenant: payee not allow-listed
spend past the budget	covenant: limit exceeded
burn it as miner fees	covenant: fees count, fee cap
fake a fresh window	covenant: wrong new state
fast-forward the clock	consensus: non-BIP68-final

Each row is a live button on the demo page: it signs the attack with the (published) agent key and broadcasts it to chipnet. The node's refusal is shown verbatim.

Benefits of AgentLeash

- **The owner sets the rules once, they become the contract address**
  - budget per window of blocks, **miner fees included**
  - up to 3 allowed payees, a fee cap per payment
- **The agent pays on its own, no server holds the money**
  - a tool answers `402`, the agent signs a covenant payment, the tool serves
  - every payment carries an on-chain receipt bound to the HTTP request
- **The network says no, even to someone holding the agent key**
- **The owner pulls the plug any time, without the agent's key**
  - withdraw, top up, migrate to new rules
- **Tiny payments stay tiny: a pay tx is 700 bytes, 705 sats ≈ 0.2 US cent at 1 sat/byte**
  - BCH ≈ 274 USD on 16.10.2026 (CoinGecko, read by the demo's own price tool)

Live demo: click, no coins, no wallet



Live page, 16.10.2026: "Run the agent" on the public demo leash, three 402 payments on chipnet in 12 s (txids a4524793…, d06f0939…, fa68b521…).

On the page (chipnet, public demo leash; its agent key is published on purpose):

1. "Run the agent": it plans, gets 402, pays each tool, txid per call
2. "Run until the leash says no": refused **before signing**
3. "Try to break it": 5 attacks, the node refuses each
4. Journal from the chain; create your own leash via WizardConnect

One command, `npm run agent:chipnet` (16.10.2026):

```
PAID #1 chain_tip   tx 51fbf483…4a37e7
PAID #2 bch_price   tx 504ade2c…73ac2c
PAID #3 fortune     tx edcc9ea0…b5dcd3
PAID #4-6 chain_tip (3 more)
NO   #7 refused by SDK: limit exceeded:
     10331 of 12000 sats spent, 1705 requested
ATTACK not allow-listed  node: mandatory-
     script-verify-flag-failed (code 16)
ATTACK 1 sat over limit  node: same (code 16)
ATTACK clock             node: non-BIP68-final
```

Planner: scripted by default; any OpenAI-compatible LLM with your key (tested with gpt-oss-20b).

BCH **Usage**

- CashScript covenant on a **P2SH32** address
- State in one **mutable NFT** (CashTokens): `elapsed | spent`, 8 + 8 bytes
- Transaction introspection fixes the shape: 1 input, back-to-contract + payment + receipt
- Clock = **proven UTXO age** via relative locktime (CSV / BIP68), never the agent's word
- Receipt: `OP_RETURN "LSH1" sha256(nonce|method|url|body)`
- Owner path: any tx that also spends the owner's P2PKH coin
- Owner wallet over **WizardConnect**; payments over x402 v2 messages, our `leash` scheme

```
function pay(pubkey agentPk, sig agentSig, int elapsedAdd) {
  // agent key, NFT category + capability, 3 outputs ...
  int outflow = tx.inputs[0].value - tx.outputs[0].value;
  require(outflow - amount <= maxFee);        // fee cap
  require(elapsedAdd >= 0 && elapsedAdd < 65536);
  require(this.age >= elapsedAdd);           // CSV: proven age
  int e = elapsed + elapsedAdd;
  int s = spent + outflow;                    // fees count
  if (e >= period) { e = 0; s = outflow; }    // window rolls
  require(s <= limit, "pay: limit exceeded");
  require(tx.outputs[0].nftCommitment ==
          toPaddedBytes(e, 8) + toPaddedBytes(s, 8));
}
```

Simplified from `contracts/AgentLeash.cash` (CashScript 0.14).

Does it work? Tested like an attacker

183

automated tests (vitest, CashScript mock network that runs the covenant in the BCH VM), incl. **29 attacks** with the exact rejection text and a property test of the worst case (2 × limit around a reset)

Real

chipnet runs with txids in the README: covenant, 402 paywall (also with 1 confirmation, a real block), web page, WizardConnect over the public relays, the agent

Unaudited

prototype; 7 automated review rounds by another vendor's model (Google Jules: contract, paywall, WizardConnect), each finding fixed with a test or documented; no professional audit yet

Traps we found and closed (each has a test):

fees outside the limit	many 546-sat payments burn several × the limit as fees → the limit counts payment + fee
CSV masking	BIP68 ignores bits above 16 and treats bit 22 as time → `elapsedAdd < 65536`
age off by one	age seen by the next block = tip + 1 − height, checked against a chipnet node
owner signature	the owner input must be `SIGHASH_ALL`, else others could rewrite a withdrawal (documented)
fake "attack" on a fresh window	(0, payment) is the honest state there → the demo does not build it (found while building the agent)

Will it make money?

Who pays, and why

- **Teams running AI agents that buy APIs and data**
  - need a hard cap they do not have to trust, plus receipts per call
- **API sellers who want agent customers**
  - a 402 paywall that settles in BCH, ~0.2 cent fee per payment
- **Wallets (Paytaca, Selene, Cashonize)**
  - "agent allowance" as a wallet feature: distribution for us, a new use for them

Revenue to test (proposed, not validated)

- **Hosted 402 gateway for API sellers: a small take rate on settled payments**
- **Managed leash for agent teams: policy templates, alerts, monitoring**
- **Open core (MIT SDK and contract) keeps the trust story checkable**

Why BCH: sub-cent fees for per-call payments, covenants and CashTokens on mainnet, no custodian in the path.

Honest status: prototype, no users, no revenue. Next: 5 conversations with agent builders before building more.

Related work, honestly

Closest prior work: **FortressVault** (BCH BLAZE 2025, 2nd place, Mainnet Track): a covenant vault with a daily withdrawal limit kept in a mutable NFT, a rescue address, a live site. Same family of mechanism (state in an NFT, per-period limit). What AgentLeash adds (left column from its public project page, 2025):

	FORTRESSVAULT (2025)	AGENTLEASH (2026)
Who spends	the owner, within a daily limit	a **separate agent key**; the owner exits without it
Fees	not checked by us	**count against the limit**, fee cap; fee drain is a tested attack
Clock	daily window (mechanism not checked)	**proven UTXO age** (CSV) + the masking bound
Payees	any address, within the daily limit	**up to 3 allow-listed**
Payments for services	not described	**402 paywall**, receipt bound to the HTTP request, agent loop
Public attack demo	not described	5 attacks broadcast, verbatim node refusal

x402 on BCH elsewhere: `@x402/bch` (x402-foundation/x402 PR #3632, open on 15.10.2026, `exact` scheme, P2PKH inputs only) and an independent x402-bch facilitator. Neither spends from a covenant; the `leash` scheme does. We have not implemented `exact`.

The Vision for AgentLeash

Every AI agent that pays gets a leash by default: the budget lives on chain, not in a prompt.

WHEN	WHAT	STATE
10–16.10	covenant, SDK, 402 paywall, WizardConnect owner path, web demo, agent	done chipnet
by 31.10	capped mainnet run (≤ 3 USD): genesis, agent payment, attack refused, withdraw	planned script ready
by 31.10	real wallet apps sign the owner path (Paytaca, Cashonize, Selene)	planned Paytaca's code: pass
31.10	contract freeze (tag), external review round	planned
November	mainnet demo leash on the site, submission	planned
next	limits in a stablecoin CashToken (e.g. PUSD), `exact` scheme, sliding window, audit	roadmap

Target: the default way an agent pays for an API on BCH. No user or revenue numbers yet; we will not invent any.

THANK YOU!

Try to steal the demo agent's money:

https://ttimesai-star.github.io/agentleash-bch/

Source, tests, chipnet logs: github.com/ttimesai-star/agentleash-bch · MIT

Prototype, unaudited. Do not put real money in it beyond the capped demo.